

剩余知识点题型适配模拟卷

目录

卷面说明	2
A 卷题目	2
B 简明答案	4

卷面说明

本卷只针对前面四套卷没有完全展开的 PPT 内容。题型按知识点适合方式安排：

1. 背景型知识点：适合简答、判断、对比。
2. 理论细节：适合解释原因、说明规则。
3. 过程型知识点：适合小计算、小推导。
4. 次级但可出题内容：适合短流程题。

A 卷题目

一、背景型简答：IR、NFA/DFA、SSA（20 分）

回答下列问题：

1. 说明高层 IR 和低层 IR 的区别，各举一个特点。
2. 说明为什么 DFA 的执行通常比 NFA 的直接模拟更简单。
3. 说明 SSA 为什么有利于优化。
4. 说明工业编译器中使用 SSA 或类似 SSA 形式的原因。

二、理论细节：SDD、SDT、属性文法（20 分）

回答下列问题：

1. 说明 SDD 和 SDT 的区别。
2. 说明属性文法和带副作用语义动作的区别。
3. 为什么属性依赖图中存在环时，属性值无法正常求值。
4. 什么是受控副作用，写出两个基本要求。

三、过程型小题：LR 表格驱动分析器（20 分）

已知某 LR 分析表有如下条目：

```
ACTION[0,id] = s1
ACTION[1,+] = r(E -> id)
ACTION[1,$] = r(E -> id)
ACTION[2,+] = s3
ACTION[2,$] = acc
ACTION[3,id] = s4
ACTION[4,$] = r(E -> E + id)

GOTO[0,E] = 2
```

输入串：

`id + id $`

要求：

1. 说明 ACTION 表和 GOTO 表分别在什么时候使用。
2. 写出分析过程中栈和输入的大致变化。
3. 说明什么时候接受输入。

四、次级流程题：通用左递归消除（20 分）

文法：

`S -> A a | b`
`A -> S d | c`

按非终结符顺序：

`S, A`

1. 判断该文法是否存在间接左递归。
2. 按通用左递归消除思路，先把 `A -> S d` 中的 `S` 用 `S` 的产生式替换。
3. 消除替换后 `A` 的直接左递归。

五、次级流程题：数组类型的 L 属性定义（20 分）

考虑数组类型声明：

`T -> B C`
`B -> int | float`
`C -> [ num ] C | epsilon`

设计一个 L 属性 SDD，用于计算类型 `T.type`。要求：

1. `B.type` 表示基本类型。
2. `C.in` 表示从左侧传入的已有类型。
3. `C.type` 表示加上数组维度后的类型。
4. 对声明形式 `int[10][20]` 写出类型构造顺序。

B 简明答案

一、背景型简答答案

高层 IR 和低层 IR:

高层 IR 保留较多源语言结构, 例如循环、函数、对象等, 便于前端语义分析和高层优化。
低层 IR 更接近机器, 例如显式跳转、加载/存储、寄存器或栈操作, 便于后端代码生成。

DFA 比 NFA 更容易直接执行:

DFA 在每个状态、每个输入符号上最多只有一个确定转移。
NFA 可能有多个转移和 `epsilon` 转移, 直接模拟时要维护一组可能状态。
因此 DFA 执行时只需要维护一个当前状态。

SSA 有利于优化:

SSA 中每个变量版本只赋值一次, 定义和使用关系清楚。
这有利于常量传播、死代码删除、公共子表达式消除等优化。

工业编译器使用 SSA 或类似形式的原因:

SSA 能简化数据流分析, 让优化算法更容易实现。
实际编译器在后端会把 `phi` 函数转换成复制、寄存器分配或其它等价低层操作。

二、理论细节答案

SDD 和 SDT:

SDD 是语法制导定义, 偏抽象, 描述文法符号有哪些属性以及属性如何计算。
SDT 是语法制导翻译, 偏实现, 把语义动作嵌入到产生式中, 说明分析过程中何时执行动作。

属性文法和带副作用语义动作:

属性文法主要通过属性之间的依赖计算语义信息, 不依赖修改全局状态。
带副作用语义动作可能会修改符号表、输出代码或更新全局结构。

依赖图中有环时无法求值:

如果属性 A 依赖属性 B, 同时 B 又直接或间接依赖 A, 就没有合法的先后计算顺序。
依赖图存在环时, 无法得到拓扑序, 因此属性值无法正常求出。

受控副作用:

受控副作用是指语义动作虽然会修改外部状态, 但执行位置和执行顺序明确, 并且不破坏属性求值。
基本要求包括:

1. 副作用执行前所需属性已经计算完成。
2. 副作用不会引入依赖环。
3. 副作用不会因分析顺序不确定而重复执行或漏执行。

三、过程型小题答案

ACTION 和 GOTO:

ACTION[state, 终结符] 用于面对输入符号时决定移进、规约或接受。
GOTO[state, 非终结符] 用于规约出非终结符之后决定进入哪个状态。

分析过程:

状态栈	符号栈	输入	动作
0	空	id + id \$	ACTION[0,id] = s1
0 1	id	+ id \$	ACTION[1,+] = r(E -> id)
0 2	E	+ id \$	GOTO[0,E] = 2
0 2 3	E +	id \$	ACTION[2,+] = s3
0 2 3 4	E + id	\$	ACTION[3,id] = s4
0 2 3 4	E + id	\$	ACTION[4,\$] = r(E -> E + id)
0 2	E	\$	ACTION[2,\$] = acc

说明:

接受条件是当前状态和输入结束符 \$ 对应的 ACTION 表项为 acc。

四、通用左递归消除答案

原文法:

$S \rightarrow A a \mid b$
 $A \rightarrow S d \mid c$

存在间接左递归:

$S \Rightarrow A a \Rightarrow S d a$

按顺序 S, A, 处理 A 时, 把 $A \rightarrow S d$ 中的 S 替换为 S 的右部:

$A \rightarrow A a d \mid b d \mid c$

此时 A 有直接左递归:

$A \rightarrow A a d \mid b d \mid c$

消除直接左递归:

$A \rightarrow b d A' \mid c A'$
 $A' \rightarrow a d A' \mid \epsilon$

最终文法:

$S \rightarrow A a \mid b$
 $A \rightarrow b d A' \mid c A'$
 $A' \rightarrow a d A' \mid \epsilon$

五、数组类型 L 属性定义答案

语义规则：

```
T -> B C
  C.in = B.type
  T.type = C.type

B -> int
  B.type = int

B -> float
  B.type = float

C -> [ num ] C1
  C1.in = array(num.val, C.in)
  C.type = C1.type

C -> epsilon
  C.type = C.in
```

对 `int[10][20]`：

```
B.type = int
C.in = int

读到 [10]:
C1.in = array(10, int)

读到 [20]:
C2.in = array(20, array(10, int))

C2 -> epsilon:
C2.type = array(20, array(10, int))
```

最终：

```
T.type = array(20, array(10, int))
```

说明：

该写法按从左到右的语义动作构造类型。

如果课程约定数组类型按相反嵌套顺序表示，卷面应按老师 PPT 中的记法调整。